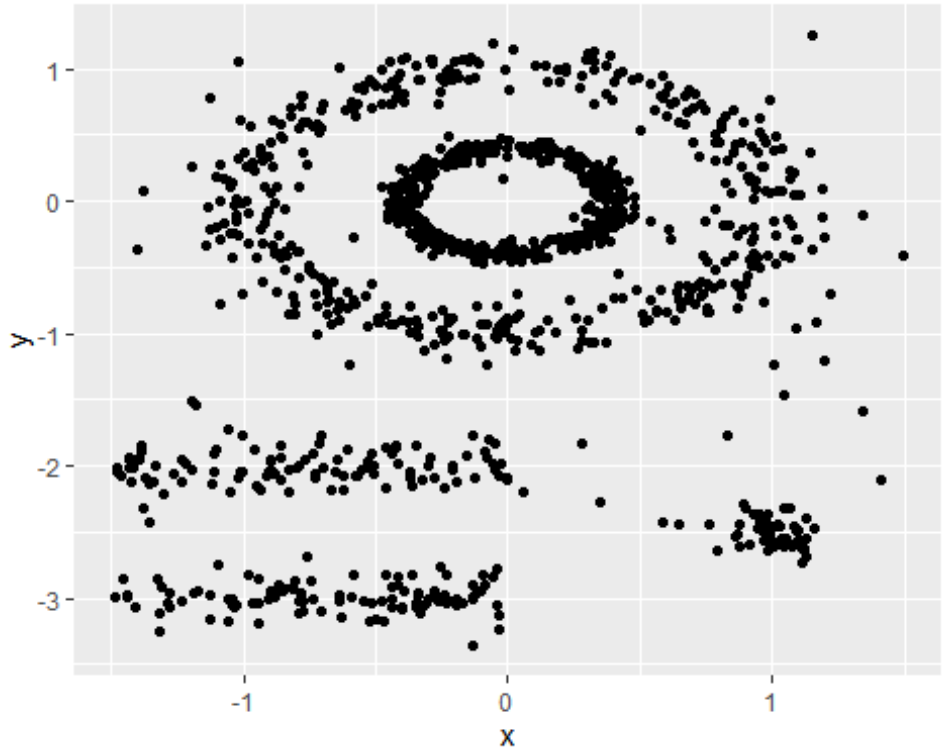
DBSCAN

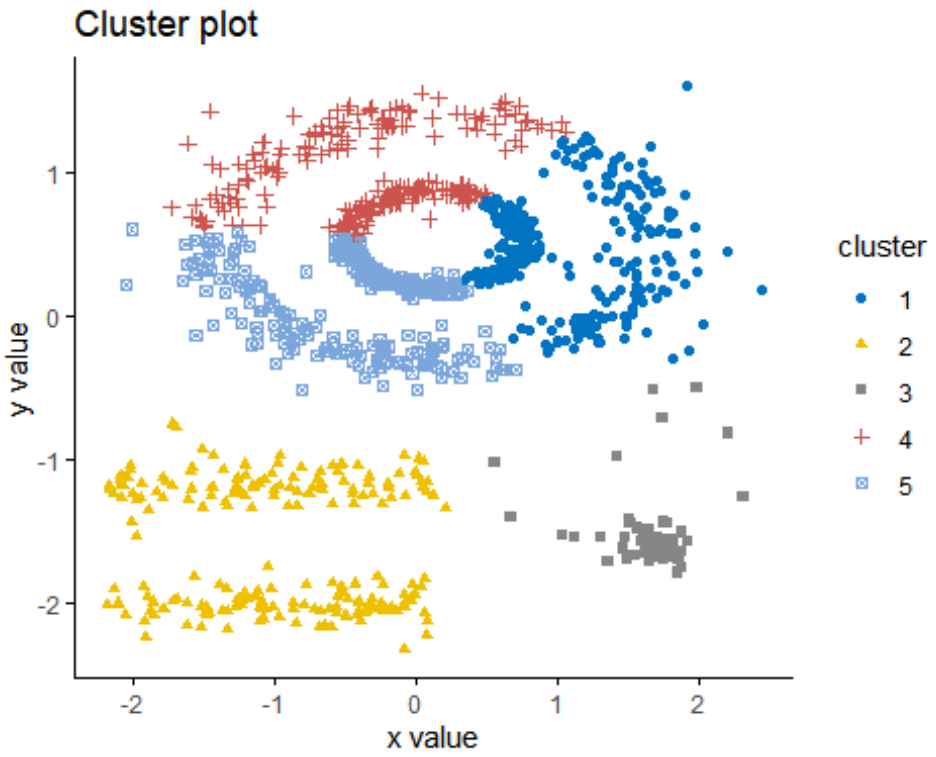
DANA 4840

```
# Load the data set
library("factoextra")

## Loading required package: ggplot2

## Welcome! Want to learn more? See two factoextra-related books at
https://goo.gl/ve3WBa

data("multishapes")
df <- multishapes[, 1:2]
# Visualize the data
ggplot(df, aes(x, y)) + geom_point()
```

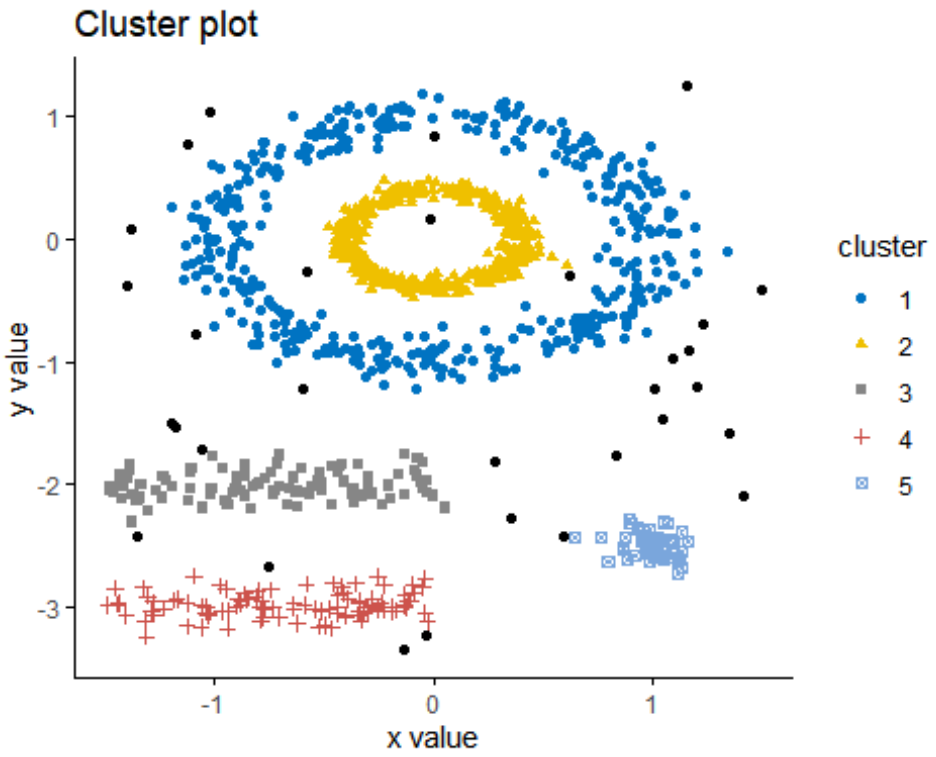


Checking K-means

```
set.seed(123)
km.res <- kmeans(df, 5, nstart = 25)
fviz_cluster(km.res, df, geom = "point",
ellipse= FALSE, show.clust.cent = FALSE,
palette = "jco", ggtheme = theme_classic())
```




```
## Compute DBSCAN using fpc package
library("fpc")
set.seed(123)
db <- fpc::dbscan(df, eps = 0.15, MinPts = 5)

# Plot DBSCAN results
library("factoextra")
fviz_cluster(db, data = df, stand = FALSE,
             ellipse = FALSE, show.clust.cent = FALSE,
             geom = "point",palette = "jco", ggtheme = theme_classic())
```



Note that, the function *fviz_cluster*() uses different point symbols for core points (i.e, seed points) and border points. Black points correspond to outliers. You can play with *eps* and *MinPts* for changing cluster configurations.

It can be seen that DBSCAN performs better for these data sets and can identify the correct set of clusters compared to k-means algorithms.

```
print(db)
```

```
## dbscan Pts=1100 MinPts=5 eps=0.15
##          0   1   2   3  4  5
## border 31  24   1   5  7  1
## seed    0 386 404  99 92 50
## total  31 410 405 104 99 51
```

In the table above, column names are cluster number. Cluster 0 corresponds to outliers (black points in the DBSCAN plot). The function print.dbscan() shows a statistic of the number of points belonging to the clusters that are seeds and border points.

```
db$cluster[sample(1:1089, 20)]
```

```
##  [1] 2 2 1 2 1 4 1 2 2 2 0 4 1 1 3 1 2 1 4 2
```

DBSCAN algorithm requires users to specify the optimal eps values and the parameter MinPts. In the R code above, we used eps = 0.15 and MinPts = 5. One limitation of DBSCAN is that it is sensitive to the choice of ϵ, in particular if clusters have different densities. If ϵ is too small, sparser clusters will be defined as noise. If ϵ is too large, denser clusters may be

merged together. This implies that, if there are clusters with different local densities, then a single ϵ value may not suffice.

A natural question is:

How to define the optimal value of ϵ ?

The function kNNdistplot() [in dbscan package] can be used to draw the k-distance plot:

```
dbscan::kNNdistplot(df, k =  5)
abline(h = 0.15, lty = 2)
```